e起守忆影像修复平台

面向社区老人服务的影像修复系统，包含：

- 小程序端：老人上传照片、留言需求、AI或人工修复。
- 后台管理端：任务管理、学生志愿者处理、系统运营。
- Java后端：统一鉴权、任务调度、数据管理。
- Python AI节点：Windows本地多GPU推理 + Linux vLLM部署入口。

---

1. 项目结构

- `ruoyi-app` ：uni-app 小程序端。
- `ruoyi-ui` ：Vue3 + Element Plus 后台管理端。
- `ruoyi-admin` ：Spring Boot 管理接口层。
- `ruoyi-system` ：系统与业务核心模块。
- `ruoyi-framework` ：安全、登录、注册、权限框架。
- `ruoyi-common` ：公共模型、工具、常量。
- `ruoyi-fastapi` ：AI推理服务。
- `sql/ry_eqsy_repair.sql` ：数据库初始化脚本（已按本项目改造）。

---

2. 当前关键能力

2.1 小程序端

- 适老化首页引导。
- 工作台上传照片并提交任务。
- 修复方式：AI修复 / 人工志愿者精修。
- 留言备注（提示词）输入。
- 手机号短信验证码登录/注册流程。

2.2 后台管理端

- 任务查看、进度跟踪、人工处理流转。
- Web注册支持手机号 + 短信验证码。
- 注册统一为后台学生志愿者账号（ `common` 角色）。

2.3 后端注册规则

- 后台注册统一绑定普通角色（ `common` ），用于学生志愿者处理任务。
- 管理员账号由系统管理员在后台创建，不走公开注册页。
- 注册成功后自动写入 `sys_user_role` 。

---

3. AI服务部署模式

3.1 Windows部署（本地多GPU）

入口文件： `ruoyi-fastapi/main(win).py`

适用场景：

- 本地有多张显卡（例如 2 x 5060Ti 16G）。
- 希望自动扩展到更多GPU。

特性：

- 自动识别 GPU 数量并为每张卡加载独立模型实例。
- 任务按当前负载分配到空闲GPU。
- 支持 `LOCAL` 本地模型推理， `CLOUD` 硅基流动云端回退。

安装依赖（Windows）：

```
cd E:\外包工作\江理工小程序\ruoyi-fastapi
python -m pip install -r requirements.txt
```

按 CUDA 单独安装 torch（示例，CUDA 12.8）：

```
python -m pip install torch torchvision --index-url https://download.pytorch.org/whl/cu
```

说明：

- torch 需要与你机器 CUDA 版本匹配，不建议写死在 `requirements.txt` 。
- xformers 为可选优化库，不装也可运行。

启动示例（PowerShell）：

```
cd E:\外包工作\江理工小程序\ruoyi-fastapi
$env:WIN_FASTAPI_PORT="8001"
$env:WIN_EXECUTOR_WORKERS="8"
$env:SILICONFLOW_API_KEY="<your_api_key>"
python "main(win).py"
```

3.2 Linux部署（vLLM本地AI口）

入口文件： `ruoyi-fastapi/main(Linux).py`

适用场景：

- 服务器部署（多GPU）
- vLLM 或 OpenAI-compatible 本地AI服务统一接入

特性：

- 支持多GPU本地推理调度（LOCAL）并启用懒加载。
- 支持 `CLOUD/VLLM` 路由到 vLLM/OpenAI-compatible 接口。
- 当选择云端模式时，不加载本地模型，不占本机显存。

安装依赖（Linux）：

```
cd /data/eqsy/ruoyi-fastapi
python3 -m pip install -r requirements.txt
```

按 CUDA 单独安装 torch（示例，CUDA 12.8）：

```
python3 -m pip install torch torchvision --index-url https://download.pytorch.org/whl/c
```

仅 CPU 环境（示例）：

```
python3 -m pip install torch torchvision
```

启动示例（Linux）：

```
cd /data/eqsy/ruoyi-fastapi
export LINUX_FASTAPI_PORT=8001
export VLLM_BASE_URL=http://127.0.0.1:8000
export VLLM_IMAGE_EDIT_PATH=/v1/images/edits
export VLLM_API_KEY=<your_vllm_key_if_needed>
python "main(Linux).py"
```

4. 安全说明

- 不要把 API Key 写入代码仓库。
- 使用环境变量：
  - `SILICONFLOW_API_KEY`
  - `VLLM_API_KEY`
- 若密钥曾在聊天或截图中暴露，请立即在服务商控制台轮换。

4.0 当前临时密钥状态（仅测试）

- 目前 `ruoyi-fastapi/main(Win).py` 与 `ruoyi-fastapi/main(Linux).py` 已内置临时回退密钥（仅用于当前阶段联调）。
- 若环境变量已设置，则环境变量优先，会覆盖代码中的临时回退值。
- 上线前必须移除代码内置密钥，仅保留环境变量注入。

4.1 硅基流动 API Key 修改位置（重点）

本项目调用硅基流动时读取环境变量 `SILICONFLOW_API_KEY` 。

代码读取位置：

- `ruoyi-fastapi/main(Win).py` 中通过 `os.getenv("SILICONFLOW_API_KEY", "")` 读取。
- `ruoyi-fastapi/main(Linux).py` 中通过 `os.getenv("VLLM_API_KEY", "")` 读取。

也就是说：

- 你不需要改 Python 源码里的常量字符串。
- 只需要在启动 `ruoyi-fastapi` 的机器上修改环境变量，然后重启 AI 服务。

Windows（PowerShell）临时生效（仅当前终端）：

```
cd E:\外包工作\江理工小程序\ruoyi-fastapi
$env:SILICONFLOW_API_KEY="你的新密钥"
python "main(Win).py"
```

Windows（PowerShell）持久生效（后续新终端可用）：

```
setx SILICONFLOW_API_KEY "你的新密钥"
```

Linux 临时生效（当前 shell 会话）：

```
export SILICONFLOW_API_KEY="你的新密钥"
python "main(Linux).py"
```

Docker / Compose：

- 在 `eqsy-ai` 服务的 `environment` 中增加或更新： `SILICONFLOW_API_KEY=你的新密钥` 。
- 变更后重建/重启容器使其生效。

生效验证：

- 重启 AI 服务后发起一次 `provider=CLOUD` 的修复请求。
- 若密钥为空或错误，服务通常会返回鉴权失败（如 401/403）或提示缺少 `SILICONFLOW_API_KEY` 。

---

5. 关键配置项

数据库配置表 `sys_config` 已使用：

- `sys.account.registerUser=true` ：开启注册。

对应 SQL 位置： `sql/ry_eqsy_repair.sql`

---

6. 接口摘要

登录注册

- `POST /sendSmsCode` ：发送短信验证码（当前为开发模拟发送，可对接真实短信网关）。
- `POST /smsLogin` ：短信验证码登录。

- `POST /register` ：注册（学生志愿者，支持 `smsCode` ）。

AI修复

- `POST /api/ai/repair/basic`
  - 参数： `file` ， `prompt` ， `task_id` ， `provider`
  - `provider=LOCAL|CLOUD` （Windows版本）
  - Linux版本统一走 vLLM 口

小程序“第3步备注”到模型提示词链路

- 小程序工作台第3步输入会写入任务字段 `remark` （ `ruoyi-app/pages/work/index.vue` 里 `remark: this.aiPrompt` ）。
- FastAPI 接口支持 `prompt` 参数并会用于本地与云端推理。
- Java 侧现已改为真实 HTTP 网关，会调用 FastAPI `/api/ai/repair/basic` 。
- `remark` 会作为 `prompt` 传给模型， `taskType` 会映射为 `provider` （ `LOCAL/CLOUD/VLLM` ）。

Java 网关配置位置：

- `ruoyi-admin/src/main/resources/application.yml`
  - `ai.fastapi.base-url` ：FastAPI服务地址
  - `ai.fastapi.repair-path` ：修复接口路径

调试日志建议：

- 在 `application.yml` 中开启
  - `logging.level.com.ruoyi.system.service.impl.MockLocalAiRepairGateway=debug`
- 关键观察日志：
  - `ai-gateway start` （任务开始）
  - `ai-gateway fastapi response` （FastAPI状态码）
  - `ai-gateway success` （耗时与结果URL）

---

7. 开发建议

1. 本地联调时先启 Java 后端，再启 `ruoyi-app` / `ruoyi-ui` ，最后启 `ruoyi-fastapi` 。

2. 如微信开发者工具出现 websocket 告警，优先排查开发工具网络与域名校验设置，不要先误判为业务代码崩溃。
3. 生产环境建议：
    - AI服务与业务后端分机部署
    - Redis 独立实例
    - API网关限流与审计

---

8. 最近改造记录（摘要）

- 小程序首页适老化重构。
- 工作台上传/留言/结果刷新流程重构。
- 小程序短信登录注册链路联通。
- Web注册统一为学生志愿者通道（手机号+短信验证码）。
- FastAPI改为双入口：
    - `main(win).py` ：Windows多GPU版
    - `main(Linux).py` ：Linux多GPU懒加载 + vLLM路由版
- 根README改写为项目专用文档。

---

9. 服务器部署（Docker / K8s）

9.1 Docker 部署建议

推荐拆分为 4 个容器：

- `eqsy-java` ：Spring Boot 后端。
- `eqsy-ui` ：ruoyi-ui 前端静态资源（Nginx）。
- `eqsy-ai` ：Python AI 服务（Linux 用 `main(Linux).py` ）。
- `redis` ：缓存与短信验证码存储。

可参考最小 `docker-compose.yml` （示例结构）：

```yaml
version: '3.8'
services:
  redis:
    image: redis:7
    ports:
      - "6379:6379"

  eqsy-java:
    image: your-registry/eqsy-java:latest
    environment:
      SPRING_PROFILES_ACTIVE: prod
      REDIS_HOST: redis
    depends_on:
      - redis
    ports:
      - "8080:8080"

  eqsy-ui:
    image: your-registry/eqsy-ui:latest
    ports:
      - "80:80"

  eqsy-ai:
    image: your-registry/eqsy-ai:latest
    environment:
      LINUX_FASTAPI_PORT: 8001
      LINUX_MODEL_DIR: /models/Qwen-Image-2512
      VLLM_BASE_URL: http://vllm:8000
    ports:
      - "8001:8001"
    deploy:
      resources:
        reservations:
          devices:
            - capabilities: [gpu]
```

说明：

- 生产建议使用 `nvidia-container-toolkit` 让 AI 容器可见 GPU。
- AI 模型目录通过 volume 挂载，不要打进镜像层。

9.2 Kubernetes 部署建议

建议拆分命名空间和工作负载：

- `Deployment` ： `eqsy-java` 、 `eqsy-ui` 、 `eqsy-ai` 。

- `StatefulSet` ： `redis` （或直接使用云 Redis）。
- `Service` ：分别暴露 java/ui/ai 内部访问。
- `Ingress` ：统一域名路由（如 `/api` 到 Java， `/ai` 到 AI 网关）。

GPU 关键点：

- 节点需安装 NVIDIA device plugin。
- 在 `eqsy-ai` Pod 中声明 `resources.limits.nvidia.com/gpu: 1` （或更多）。
- 若多卡节点，建议给 `eqsy-ai` 设置独立 nodeSelector/taint 容忍，避免与业务 Pod 争抢资源。

最小化 AI Deployment 资源示例：

```
resources:
  limits:
    cpu: "8"
    memory: 24Gi
    nvidia.com/gpu: 2
  requests:
    cpu: "4"
    memory: 12Gi
```

---

10. 小程序上架流程（微信）

10.1 上架前准备

- 微信小程序主体已完成认证（企业/组织）。
- 配置合法域名（request/uploadFile/downloadFile/websocket）。
- 准备隐私政策、用户协议、客服联系方式。
- 检查敏感权限说明（相册、相机、保存到相册）。

10.2 构建与提审

1. 在 HBuilderX 使用“发行 -> 小程序-微信”。
2. 在微信开发者工具上传代码，填写版本号与项目备注。
3. 提交审核前完成：
    - 账号登录与注册流程可用
    - 主要页面无明显空白/错位
    - 核心链路可演示（上传 -> 提交 -> 结果查看）

10.3 发布与灰度

- 审核通过后先走灰度发布（建议 5%-20% 用户）。
- 观察错误日志与性能后再全量发布。
- 每次版本发布前备份数据库和配置快照。

10.4 常见卡点

- `websocket/域名` 报错：通常是开发者工具与合法域名配置不一致。
- 图片无法加载 ：确认 CDN/对象存储跨域与 HTTPS 证书链完整。
- 审核驳回 ：重点检查隐私声明、采集目的和用户授权文案。